ОТЧЁТ

Лабораторные работы № 5–8

Система учёта заявок

Журнал применения ИИ: проектирование, ревью и проверка

Дисциплина: Технологии разработки программного обеспечения

Студент: Дельянов Радислав

Группа: _____

Дата: 11 октября 2026 г.

Материалы подготовлены на основе проекта и проверок, перечисленных в отчёте. Публичное развёртывание пропущено по решению студента.

Журнал фиксирует рабочие запросы и результаты в проекте. Внешние диалоги не экспортировались; записи ниже описывают взаимодействия в Codex и проверки, фактически выполненные в репозитории.

Лабораторная № 5

Проектирование модели данных

Дата / этап: 11 октября 2026 г., анализ предметной области

Задача: Сопоставить сущности системы заявок с реляционными таблицами и определить связи/ограничения.

Модель: Codex — модель/версия не указаны интерфейсом

Запрос 1 - проектирование

«По ТЗ системы учёта заявок предложи нормализованную реляционную модель для пользователей, заявок, категорий, статусов, комментариев и аудита. Укажи PK/FK, обязательность связей, уникальность и CHECK-ограничения. Сохрани учебную простоту и перечисли допущения».

Запрос 2 - проверка согласованности

«Сверь ER-модель и SQLite DDL с существующим app/db.py. Найди расхождения таблиц, типов связей, ограничений и индексов. Не утверждай, что схема корректна, пока не предложишь проверяемые SQLite-инварианты».

Контекст

ТЗ docs/TZ.md, существующие app/db.py и структура проекта; передавались только учебные сущности.

Результат

За основу приняты шесть сущностей: users, categories, statuses, requests, comments и request_history; схема вынесена в DDL и ER-диаграмму.

Проверка

DDL исполнен в SQLite in-memory; проверены таблицы, PRAGMA foreign_key_check и отклонение некорректного внешнего ключа.

Доработка / решение студента

Схема сопоставлена с приложением; мягкое архивирование оставлено как поведение API, а не физическое удаление.

Почему выбран такой подход

Контекстное проектирование полезнее общего SQL-ответа; окончательное решение проверено исполняемой СУБД.

Лабораторная № 6

Планирование функциональности и ревью сервисного слоя

Дата / этап: 10 октября 2026 г., реализация

Задача: Разложить CRUD, статусы, поиск, валидацию и архивирование по слоям.

Модель: Codex — модель/версия не указаны интерфейсом

Первый запрос

«Сделай CRUD заявок на Python».

Недостатки первого запроса

Не заданы проект, роли, SQLite, сервисный слой, правила статусов, формат ошибок и критерии проверки; ответ мог бы быть слишком общим.

Уточнённый запрос

«Выступи разработчиком существующего Python-проекта “Система учёта заявок”. Составь план CRUD заявки с SQLite и repository/service слоями, обязательными полями, поиском, фильтрами, назначением исполнителя, мягким удалением и допустимыми переходами статуса. Не добавляй реальные секреты; выдай задачи и критерии проверки».

Запрос ревью

«Проверь сервисные правила переходов статусов и валидацию заявок по ТЗ. Найди обход бизнес-правил, некорректные ссылки и ошибки API; каждое замечание подкрепи конкретным тестом».

Контекст

ТЗ, архитектурное описание, app/db.py, repository.py, services.py, schemas.py.

Результат

Сформированы разделение DB/repository/service, ограничения данных, статусные переходы и тестовые данные.

Проверка

Реализация сопоставлена с требованиями; сценарии API вошли в tests/test_api.py. Четыре теста ранее прошли локально.

Доработка / решение студента

Приняты мягкое архивирование, история и явные переходы; рекомендации проверялись в коде, а не копировались безусловно.

Почему выбран такой подход

Контекст проекта и тестируемые критерии сокращают риск получить посторонний каркас вместо продолжения существующей системы.

Лабораторная № 7

Проектирование и критический разбор REST API

Дата / этап: 10 октября 2026 г., проектирование API

Задача: Определить маршруты заявок и справочников, JSON-схемы, HTTP-коды, фильтры и формат ошибок.

Модель: Codex — модель/версия не указаны интерфейсом

Запрос проектирования

«Предложи REST-контракт для Request в FastAPI: список/карточка, POST, PATCH, DELETE/archive, справочники users/statuses/categories, фильтры q/status/category/priority, Pydantic-валидация и коды 200/201/204/404/422. Отдели endpoint от service и не включай реальные секреты».

Запрос ревью

«Критически проверь API-контракт: обязательные поля, диапазоны, HTTP-коды, единообразие ошибок, безопасность и отсутствие дублирования бизнес-логики. Для каждого замечания предложи способ воспроизводимой проверки».

Контекст

app/main.py, app/schemas.py, требования ЛР № 7; демонстрационные идентификаторы.

Результат

Подготовлены CRUD endpoint-ы, справочники, фильтры, OpenAPI и единый формат ошибок.

Проверка

docs/requests.http и Postman collection содержат сценарии успешных и ошибочных запросов; автотесты проверяют ключевые API-пути.

Доработка / решение студента

Оставлены ошибки 404/422 по смыслу операции; DELETE выполняет мягкое архивирование; аутентификация честно указана как ограничение.

Почему выбран такой подход

Раздельный запрос проектирования и критического ревью позволяет проверить контракт с разных сторон.

Лабораторная № 8

Docker-конфигурация и локальный контейнер

Дата / этап: 10–11 октября 2026 г., подготовка и ручная проверка

Задача: Подготовить повторяемый Docker-запуск и проверить пользователя контейнера, контекст сборки, порт, секреты и health-check.

Модель: Codex — модель/версия не указаны интерфейсом

Запрос генерации

«Предложи Dockerfile для Python 3.12 FastAPI + SQLite: app/main.py, requirements.txt и prototype, запуск на 8000, настройки через окружение и /health. Исключи секреты, локальную БД и виртуальное окружение из build context».

Запрос ревью

«Проведи ревью Dockerfile и Compose: права процесса, базовый образ, слой зависимостей, контекст, volume SQLite, согласованность портов, отсутствие обязательного несуществующего .env и работоспособность health-check. Предложи минимальные исправления».

Контекст

Dockerfile, .dockerignore, docker-compose.yml, .env.example; секреты и реальные пользовательские данные не передавались.

Результат

Образ Python 3.12 slim, непривилегированный пользователь, Compose volume и health-check. Compose изменён так, чтобы локальный запуск не требовал отсутствующий .env.

Проверка

По скриншотам пользователя 11.10.2026 образ собран, контейнер healthy, localhost:8000 опубликован; /health вернул status=ok и database=ok. Удалённый сервер не проверялся.

Доработка / решение студента

Учтены пути COPY и изоляция данных; выбран локальный Docker-деплой. Публичный деплой пропускается по решению студента.

Почему выбран такой подход

Ограниченный контекст и проверка запуском отделяют полезные инфраструктурные предложения от общих шаблонов; модель выбрана как помощник для технического ревью.